

WHAT I AUTOMATE

반복되는 인프라 작업을 책임질 수 있는 자동화로 바꿉니다.

배포와 인프라 운영에서 사람이 계속 반복해야 하는 일을 발견하면 더 빠르게 사용할 수 있는 도구로 만듭니다. 그리고 자동화가 실패했을 때 무엇을 지켜야 하는지까지 함께 설계합니다.

“반복하는 작업이 싫습니다.
그 일을 자동화하는 것이 개발의 본질에
가깝다고 생각합니다.”

약 5분

Heimdall · 저장소 등록부터 외부 URL까지

5개 작업

Gjallar · 실제 Proxmox 환경에서 검증

End-to-end

K-Le-PaaS · 자연어 요청에서 Kubernetes 실행까지

OPERATING PRINCIPLE

자동화는 클릭 수를 줄이는 것에서 끝나지 않습니다.

귀찮음을 출발점으로 삼되, 반복을 없앤 뒤 생기는 새로운 책임까지 구현 범위로 생각합니다.

01

반복을 먼저 찾습니다.

사람이 매번 복사하고 확인하고 연결하는 구간을 찾아 자동화의 경계로 삼습니다.

02

입력은 적게, 흐름은 하나로 만듭니다.

사용자가 알아야 할 세부 명령보다 원하는 결과에 집중할 수 있도록 실행 과정을 연결합니다.

03

성공의 기준을 실제 상태에 둡니다.

요청 접수에서 끝내지 않고 결과를 다시 확인하며 실패할 때 지켜야 할 상태를 정합니다.

AUTOMATION LOOP

REPEAT

반복되는 일 발견

ABSTRACT

필요한 입력만 남김

AUTOMATE

하나의 흐름으로 연결

VERIFY

실제 상태로 결과 확인

PLATFORM ENGINEER로서 만들고 싶은 것

개발자가 인프라의 세부 절차보다 자신의 제품에 집중하도록 돕는 신뢰 가능한 셀프서비스 플랫폼

01

HEIMDALL

PERSONAL · AI-ASSISTED

외부 배포의 반복을 하나의 요청으로 줄였습니다.

약 5분

Docker 빌드 포함

PROBLEM

외부에 띄우기까지의 과정이 매번 귀찮았습니다.

소스 준비, Docker 이미지 빌드, 컨테이너 실행, 상태 확인과 공개 경로 연결을 프로젝트마다 반복해야 했습니다.

SOLUTION

저장소 등록 이후의 배포 흐름을 연결했습니다.

서비스 구성을 한 번 등록하면 commit 선택부터 빌드, 실행, 상태 확인과 Preview 연결까지 수행하는 셀프 호스팅 도구를 만들었습니다.

RESULT

저장소에서 공개 URL까지 약 5분 안에 연결했습니다.

DNS와 공개 Edge가 준비된 홈랩에서 저장소 등록부터 외부 URL 응답까지 Docker 빌드를 포함해 검증했습니다.

BEFORE

소스 → 빌드 → 실행 → 확인 → 공개 경로 연결

AFTER

저장소와 설정 등록 → 배포 요청 → 공개 URL

AUTOMATION WITH RESPONSIBILITY

새 배포를 별도 candidate로 검증하고, 실패하면 기존 정상 Preview를 유지하도록 설계했습니다.

github.com/CodingPenguin-yoon/heimdall_final ↗

02

GJALLAR

PERSONAL · AI-ASSISTED

반복하던 VM 작업을 웹의 셀프서비스 흐름으로 바꿨습니다.

5개

실제 작업 검증

PROBLEM

VM 한 대를 만들기 전후로 확인할 것이 많았습니다.

노드와 가용 리소스를 일일이 확인하고, 생성 이후의 작업도 Proxmox 화면에서 각각 수행해야 했습니다.

SOLUTION

리소스 확인과 VM 생명주기를 하나의 화면에 모았습니다.

노드 상태와 가용 리소스를 보여주고, 원하는 사양을 입력해 VM을 생성하고 관리하는 웹 도구를 만들었습니다.

RESULT

실제 Proxmox 홈랩에서 모든 핵심 흐름을 확인했습니다.

VM 생성, 시작, 종료, 잠금 해제와 삭제까지 다섯 작업을 end-to-end로 검증했습니다.

01	02	03	04	05
CREATE	START	SHUTDOWN	UNLOCK	DELETE

AUTOMATION WITH RESPONSIBILITY

API 요청 접수만 성공으로 보지 않고 task 종료와 실제 VM 상태를 다시 확인하도록 만들었습니다.

github.com/CodingPenguin-yeon/Gjallar

03

K-LE-PAAS

TEAM OF 2 · COMPETITION PROTOTYPE

Kubernetes 명령어의 진입장벽을 자연어 흐름으로 낮췄습니다.

E2E

실행 결과 확인

PROBLEM

초보 개발자는 작업보다 명령어부터 배워야 했습니다.

Pod 상태를 확인하거나 리소스를 변경하려면 Kubernetes 명령어와 개념을 먼저 익혀야 했습니다.

SOLUTION

자연어 요청을 확인 가능한 실행 흐름으로 바꿨습니다.

자연어를 Kubernetes 작업으로 변환하고, 실행 전 확인과 실행 결과 반환까지 연결했습니다.

RESULT

NKS 환경에서 실제 상태 변화를 시연했습니다.

조회, 롤백, 재시작과 스케일링을 수행하고 터미널에서 Kubernetes 상태 변화를 확인했습니다.

MY ROLE

인프라 작업 전반
백엔드 자연어 변환

SCOPE

2인 공모전 프로토타입입니다. 심사 현장에서 Pod 상태 조회가 한 차례 사용됐으며, 지속적인 실사용 사례는 아닙니다.

AUTOMATION WITH RESPONSIBILITY

자연어를 바로 실행하지 않고 사용자가 해석된 작업을 확인한 뒤 실행하도록 구성했습니다.

youtube.com/watch?v=tY4XmxIsDok · 12-minute demo ↗

THE WORK I WANT TO CONTINUE

팀이 반복하는 일을 발견하고,
믿고 맡길 수 있는 플랫폼으로
만들겠습니다.

WHAT I BRING

- | | |
|----|--|
| 01 | 실제 불편에서 출발해 자동화할 문제를 구체적으로 정의합니다. |
| 02 | 복잡한 인프라 절차를 개발자가 사용할 수 있는 흐름으로 바꿉니다. |
| 03 | 속도뿐 아니라 검증, 실패, 기존 상태 보존까지 성공 기준에 포함합니다. |

PROJECT INDEX

- | | |
|----|---|
| 01 | Heimdall
Deployment automation |
| 02 | Gjallar
Proxmox operations |
| 03 | K-Le-PaaS
Natural language operations |

CONTACT

조운호 · Platform Engineer

code.penguin.yoon@gmail.com
github.com/CodingPenguin-yoon
[yoonman.page](#)